



我想要仔細複習 系統設計與系統分析 的各種細節

，包含各種該怎麼設計軟體系統、該怎麼做系統分析的概念(例如什麼時候kafka、redis、rebbitMQ，什麼時候做Consumer-Producer...等)

我要你列出細節，不要模糊帶過，我會依照你這份重點，作為複習的主題，逐一複習

在你回覆剛剛那三個小問題之前，先給你一份「可直接拿來當複習主線」的系統設計／系統分析大綱草稿，等你選好側重範圍之後，可以再一起微調、加深某些區塊。

下面的目標是：

- 粗體是大章節 = 你可以當成「一個晚上」或「一兩天」的主題
- 子項目儘量具體（像題目）= 你可以逐條查／設計／畫圖練習
- 會用直觀→再用專業語言的方式寫，但先列大綱，不全部展開細節推導（細節之後可逐章展開）

0. 系統設計 & 系統分析 的核心觀念

1. 問題定義與邊界

- 功能需求 vs 非功能需求 (Functional vs Non-Functional Requirements)
- SLA / SLO / SLI：延遲、吞吐量、可用性、耐久性、RPO / RTO
- 業務邏輯邊界：什麼在系統內、什麼交給外部系統

2. 架構層級與抽象

- 高階架構圖：Client / API Gateway / Service / DB / Cache / MQ
- 邏輯架構 vs 部署架構 vs 資料流架構
- 分層設計：Presentation / Application / Domain / Infrastructure

3. 系統設計的標準流程（你設計任意系統都可以照這幾步）

- 需求澄清 → 粗估規模 (QPS、資料量、成長率) → 高階架構 → 核心元件選型
- 深入細部元件：儲存、快取、MQ、排程、共識/一致性機制
- 瓶頸分析 & 擴展策略 → 容錯 & 災難復原 → 安全與觀測性

1. 工作負載分析與容量規劃

1. Workload 型態分類

- 線上 OLTP vs 批次/離線 OLAP
- CPU-bound / IO-bound / Latency-sensitive / Throughput-oriented
- 寫多讀少 vs 讀多寫少 vs 混合

2. 流量模型與估算

- QPS / RPS、峰值 vs 平均、突發流量模型
- 背壓 (Backpressure) 的必要性與策略
- 容量預估公式與簡單 rule of thumb (例如：單機處理量 → 需要幾台)

3. 效能目標拆解

- 延遲分佈 (p50/p95/p99)
- 吞吐量 vs 資源使用率 (CPU / Memory / Disk / Network)
- 反壅塞策略：排隊、降級 (Degradation)、限流 (Rate limiting)

2. 併發模型與 Consumer-Producer 模式

1. 為什麼要用 Producer-Consumer ?

- 解耦：生產與消費速度不一致
- 緩衝：吸收突發流量，平滑後端壓力
- 控制資源：用消費者數量控制併發度

2. 何時使用 Producer-Consumer ?

- 請求處理時間不穩定或偏長
- 後端資源 (DB、外部 API) 容易被打爆
- 允許非同步處理 / 最終一致，而非立即回應

3. 實作面細節

- 單機：bounded queue + worker threads (典型 blocking queue 模式)
- 分散式：用 MQ / Stream 系統作為 queue (Kafka / RabbitMQ / Redis Streams)
- Backpressure 設計：queue 滿了怎麼辦？丟錯誤？丟棄？降級？暫停接單？

4. Consumer-Producer 的調參與分析

- Consumer 數量 vs 併發度 vs 資源瓶頸
- 任務粒度與批次處理 (batch size 對吞吐與延遲的影響)
- 任務重試、去重 (idempotency)、死信佇列 (DLQ)

3. 訊息系統選型：Kafka vs RabbitMQ vs Redis (Stream / PubSub)

這區塊你明顯說想要加強，所以拆得比較細。

1. Redis (含 Redis PubSub / List / Stream)

- 適合用途
 - 快取為主 (低延遲存取 key-value)
 - 簡單 queue / 延遲 queue (用 List / Stream)
 - 較少主題、消息量相對中小、對 message durability 要求不極端
- 何時用 Redis 當訊息系統？
 - 任務少、邏輯簡單、對順序要求不那麼嚴苛
 - 想共用既有 Redis cluster，不想多一套系統
- 限制與風險
 - 非專門為高吞吐 log streaming 設計
 - Redis PubSub 無持久化/重播；Redis Stream 較適合小到中負載

2. RabbitMQ (AMQP)

- 核心特點
 - 支援多種 routing pattern：direct / topic / fanout / headers
 - 適合複雜路由、企業整合、命令/事件分發
 - 針對「每則 message 準確送到對的 consumer」的可靠投遞语义
- 適用場景
 - 需要豐富 routing / exchange 模式 (例如同一則訊息送給多個服務不同 queue)
 - 任務型系統 (task queue)：注意 ack、重試、DLQ 策略
- 限制
 - 長期儲存 event log 不適合，偏「短命令型 message」
 - 高吞吐長期 log streaming 不如 Kafka

3. Kafka

- 設計哲學
 - 分散式 commit log：append-only、可重播
 - 透過 partition 提供水平擴展與訂閱群組 (consumer group)
- 適用場景
 - 高吞吐事件流：log aggregation、行為事件、metric、stream processing
 - 需要重播、事件溯源 (event sourcing)、多個下游消費者
- 關鍵概念
 - Topic / Partition / Offset / Consumer Group / ISR / Replication factor
 - 至少一次 / 至多一次 / 準確一次 (exactly-once) 設計要點

4. 三者選型 decision points (你可以當作之後系統分析 checklist)

- 每秒 message 數量級？(1e2、1e4、1e6？)
- 是否需要長期保留 & 重播？
- 訊息路由是否很複雜？
- 數據主要是狀態同步？事件流？任務排程？
- 容忍的延遲、可接受的部署與維運成本

4. 儲存系統選型與資料建模

1. 資料模型

- 關聯式 (SQL) vs NoSQL (Key-Value, Document, Wide-Column, Graph)
- 正規化 vs 去正規化：讀寫 pattern 對 schema 設計的影響
- 熱資料 / 冷資料 / 歷史資料分層 (Hot / Warm / Cold storage)

2. 一致性與交易

- ACID / 事務隔離級別
- CAP 理論 / PACELC 視角：一致性 vs 可用性 vs 延遲
- 強一致 vs 最終一致：對應哪些業務需求不能放寬

3. 水平擴展與分片

- Sharding key 選擇策略
- Secondary index / Global index 的代價
- 跨分片 join / 事務處理模式

4. 缓存策略與 cache 圖樣

- Read-through / Write-through / Write-back / Cache-aside
- Cache invalidation 難點與策略 (TTL、版本號、雙寫、延遲失效)
- 熱點 key、穿透 (cache miss storm)、雪崩 (massive invalidation) 問題處理

5. 系統整體架構風格與拆分策略

1. Monolith vs Microservices

- 單體的優點／缺點 (部署簡單 vs 模組耦合、可擴展性差)
- 微服務拆分維度：bounded context、業務能力、團隊邊界
- 分散式之後帶來的成本：網路、觀測、資料一致性、交易拆解

2. 同步 vs 非同步 通訊

- HTTP / gRPC vs MQ / Event
- 何時用 request-response，何時用 event-driven
- 避免同步呼叫鏈變成「分散式 monolith」

3. API 設計與版本控制

- REST / RPC / GraphQL 選型考量
- 向後相容 (backward compatibility) 與資料演進 (schema evolution)
- Idempotency key、重試保證

4. Domain-Driven Design (DDD) 作為系統分析工具

- Ubiquitous language / Bounded Context / Aggregate / Entity / Value Object
- 事件風暴 (event storming) 幫助拆分 domain & service

6. 一致性、交易與分散式協議

1. 一致性模型

- Strong / Eventual / Causal / Read-your-write
- 哪些業務必須強一致 (例如餘額、持倉)、哪些可以最終一致

2. 分散式交易模式

- 本地交易 + 事件通知
- Saga pattern : choreography vs orchestration
- TCC (Try-Confirm-Cancel) 概念與適用範圍

3. 分散式協議概覽

- 共識 : Paxos / Raft (只要理解高階直覺就足夠多數面試)
- Leader election、心跳檢測、租約 (lease) 概念
- 分散式鎖與競態條件、idempotency vs exactly-once illusion

7. 可用性、高可用架構與故障處理

1. 故障模型與冗餘

- 單節點故障 vs AZ/Region 故障
- 主從 / 多主 / 多 Region 架構

2. 高可用設計

- 負載平衡 : L4 / L7 LB , health check , circuit breaker
- 重試策略與退避 (exponential backoff)
- 災難復原 : 備援資料中心、跨 Region 複寫、RPO / RTO 計劃

3. Degradation & Failover 策略

- 限流 / 熔斷 / 降級服務 (只給部分功能)
- 災難情況下維持核心路徑 (例如只允許查詢, 不允許下單)

8. 可靠性、重試、錯誤處理與觀測性

1. 錯誤分類

- 可重試 vs 不可重試
- 臨時性錯誤 vs 永久性錯誤

2. 重試與冪等

- 冪等操作設計 (idempotency key、自然冪等的 API 設計)
- 線上重試 vs 後台重試 (例如重試佇列、延遲佇列)

3. 觀測性 (Observability)

- Logs / Metrics / Traces
- 指標設計：系統級 (CPU, Memory, disk IO) , 服務級 (QPS, error rate, latency)
- 分散式追蹤：trace id / span id / propagation

4. Alerting 與 SLO

- 以用戶體驗為中心的 SLI (request success rate, latency)
- Error budget 與釋出節奏

9. 安全、認證授權與資料保護

1. 認證 (Authentication)

- Session / JWT / OAuth2 / OpenID Connect 基本概念
- 單點登入 (SSO)

2. 授權 (Authorization)

- RBAC / ABAC
- 權限模型設計與快取策略

3. 資料保護

- 傳輸加密 (TLS)、儲存加密
- 隱私資料與脫敏策略、審計 log

10. 系統分析方法論 (較「工程流程」視角)

1. Use Case / User Story 分析

- 典型場景路徑：正常流、例外流、錯誤流
- 底層資源參與情況 (DB、MQ、第三方服務)

2. 流程圖、序列圖、狀態機

- 序列圖：跨服務互動與同步/非同步界線
- 狀態轉移圖：訂單、交易、工作任務生命週期

3. 負載與壓力分析

- 算 QPS、峰值、爆點
- 用 Little's Law 直觀理解： $L = \lambda W$ （系統中請求數 = 到達率 × 平均停留時間）

4. 風險分析與 Failure Mode

- 單點故障盤點
- 外部依賴失效的行為（第三方 API down, DB partial failure）

11. 典型系統設計案例題目（你可以逐題練）

每題都可以照「需求澄清 → 指標 → 粗估 → 架構 → 選型 → 瓶頸 → 演進」完整寫一次。

1. 設計一個高流量的 Timeline / Feed 系統（像 Twitter / Facebook）
2. 設計簡化版 YouTube：影片上傳、轉碼、分發、點播
3. 設計一個日誌收集與實時分析平台（log ingestion + search）
4. 設計一個實時通知系統（Email / SMS / Push 多通道）
5. 設計交易撮合系統的基礎架構（不含演算法，專注於一致性與高可用）
6. 設計任務排程與執行平台（cron-on-steroids，支援重試、依賴、觀測）

熊熊，接下來請你先回答前面工具問的三個選項（系統情境、深度型態、你最想加強的併發/MQ/快取部分）。

回覆之後，可以針對上面的大綱做兩件事：

1. 幫你「標星」：先標出對你現在職涯最重要的 20% 主題，當成第一輪複習清單
2. 對你選的重點（例如「Kafka vs RabbitMQ vs Redis Stream」、「Consumer-Producer 模式」、「交易系統設計」）展開成更細的「逐條檢查表」，每一條都帶具體判斷準則與典型陷阱。